

Métodos Kernel

Basado en PRML de Bishop, Cap. 6

Representación dual, construcción de kernels y redes RBF

Objetivo

Estudiar como se relacionan los conceptos:

modelo lineal en $\phi(\mathbf{x}) \implies$ representación dual $\implies$ kernel trick.

Conectar lo anterior con modelos de predicción local:

kernels $\implies$ redes RBF $\implies$ Nadaraya–Watson.

Contenido

- 1 Repaso: modelos lineales en espacios de características.
- 2 Representación dual de regresión regularizada.
- 3 Kernel trick y predicción por similitudes.
- 4 Kernels válidos y matriz de Gram.
- 5 Kernel polinomial y kernel gaussiano.
- 6 Redes de funciones de base radial.
- 7 Modelo de Nadaraya–Watson.

Motivación: Productos internos

En redes neuronales estudiamos modelos como:

$$y(\mathbf{x}, \mathbf{w}) = \sum_j w_j \phi_j(\mathbf{x}),$$

donde las funciones base podían ser aprendidas de forma adaptativa.

Idea

En métodos kernel no siempre construimos explícitamente $\phi(\mathbf{x})$. En lugar de eso, buscamos calcular productos internos:

$$\phi(\mathbf{x})^\top \phi(\mathbf{x}').$$

El símbolo x' representa otro punto de entrada, distinto en general de x .

Algunas preguntas son: **¿por qué considerar productos internos?** **¿qué podemos hacer si solo conocemos estos productos internos?**

Motivación: Modelos basados en memoria

Un modelo paramétrico resume la información del conjunto de entrenamiento en parámetros:

$$\mathcal{D} \longrightarrow \mathbf{w}. \quad \mathcal{D} = \{(x_n, t_n)\}_{n=1}^N$$

Un modelo basado en memoria conserva los datos y predice usando similitudes:

$$y(\mathbf{x}) \approx \sum_{n=1}^N \text{similitud}(\mathbf{x}, \mathbf{x}_n) t_n.$$

Idea

Los kernels formalizan la idea de similitud, pero con una restricción matemática importante: la similitud debe comportarse como un producto interno en algún espacio de características.

Repaso: ¿Por qué productos internos miden similitud?

Una forma natural de comparar dos vectores es preguntar si apuntan en direcciones parecidas. El producto interno captura precisamente esta idea porque

$$x^\top x' = \|x\| \|x'\| \cos(\theta),$$

donde θ es el ángulo entre x y x' .

Idea

Si x y x' apuntan en direcciones similares, entonces $\cos(\theta)$ es grande y positivo, por lo que $x^\top x'$ es grande. Si son casi ortogonales, entonces $\cos(\theta) \approx 0$, y el producto interno es cercano a cero. Si apuntan en direcciones opuestas, el producto interno puede ser negativo.

Repaso: ¿Por qué productos internos miden similitud?

Cuando los vectores están normalizados, es decir, $\|x\| = \|x'\| = 1$, el producto interno se reduce a

$$x^\top x' = \cos(\theta),$$

que es justamente la similitud coseno.

Idea

Un kernel generaliza esta idea al medir productos internos después de transformar los datos:

$$k(x, x') = \phi(x)^\top \phi(x').$$

Así, $k(x, x')$ mide qué tan similares son x y x' , no necesariamente en el espacio original, sino en el espacio de características.

Repaso: espacio de características

Dado $\mathbf{x} \in \mathbb{R}^D$, podemos transformarlo mediante:

$$\phi(\mathbf{x}) = \begin{pmatrix} \phi_1(\mathbf{x}) \\ \phi_2(\mathbf{x}) \\ \vdots \\ \phi_M(\mathbf{x}) \end{pmatrix}.$$

Un modelo lineal en características es:

$$y(\mathbf{x}, \mathbf{w}) = \mathbf{w}^\top \phi(\mathbf{x}).$$

Idea

El modelo es lineal en $\phi(\mathbf{x})$, pero puede ser no lineal en $\mathbf{x}$.

Ejemplo

Si:

$$\phi(x) = (1, x, x^2, x^3)^\top,$$

entonces:

$$y(x, \mathbf{w}) = \mathbf{w}^\top \phi(x) = w_0 + w_1x + w_2x^2 + w_3x^3.$$

Idea

La regresión es lineal respecto a los parámetros w_0, w_1, w_2, w_3 , pero la función resultante es cúbica respecto a x .

Repaso: Conjunto de entrenamiento

Tenemos datos:

$$\mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^N.$$

Para cada dato:

$$y(\mathbf{x}_n, \mathbf{w}) = \mathbf{w}^\top \phi(\mathbf{x}_n).$$

El objetivo es encontrar $\mathbf{w}$ tal que:

$$\mathbf{w}^\top \phi(\mathbf{x}_n) \approx t_n, \quad n = 1, \dots, N.$$

Repaso: Matriz de diseño

Definimos:

$$\Phi = \begin{pmatrix} \phi(\mathbf{x}_1)^\top \\ \phi(\mathbf{x}_2)^\top \\ \vdots \\ \phi(\mathbf{x}_N)^\top \end{pmatrix} \in \mathbb{R}^{N \times M}.$$

El vector de predicciones sobre los datos es:

$$\mathbf{y} = \Phi \mathbf{w}.$$

Idea

Cada fila de Φ contiene las características de un dato de entrenamiento.

Repaso: Mínimos cuadrados regularizados

Usaremos el funcional:

$$J(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{ \mathbf{w}^\top \phi(\mathbf{x}_n) - t_n \}^2 + \frac{\lambda}{2} \mathbf{w}^\top \mathbf{w}.$$

Idea

- El primer término mide ajuste a los datos.
- El segundo término penaliza soluciones con pesos grandes.
- $\lambda > 0$ controla el compromiso entre ajuste y regularidad.

Repaso: Prior gaussiano sobre $\mathbf{w}$

La suma de cuadrados aparece al asumir ruido gaussiano en los targets:

$$t_n = y(\mathbf{x}_n, \mathbf{w}) + \varepsilon_n, \quad \varepsilon_n \sim \mathcal{N}(0, \beta^{-1}).$$

El término:

$$\frac{\lambda}{2} \mathbf{w}^\top \mathbf{w}$$

puede interpretarse como regularización o como el negativo del logaritmo de un prior gaussiano sobre $\mathbf{w}$.

Objetivo: pasar de $\mathbf{w}$ a coeficientes sobre datos

Queremos mostrar que la solución puede escribirse como:

$$\mathbf{w} = \sum_{n=1}^N a_n \phi(\mathbf{x}_n).$$

Idea

La solución $\mathbf{w}$ vive en el subespacio generado por los vectores transformados de entrenamiento:

$$\phi(\mathbf{x}_1), \dots, \phi(\mathbf{x}_N).$$

Esta idea luego dará lugar a la **representación dual** del problema, con coeficientes duales a_n .

Ejemplo: Derivada de un término de error

Para un solo dato:

$$E_n(\mathbf{w}) = \frac{1}{2} (\mathbf{w}^\top \phi(\mathbf{x}_n) - t_n)^2.$$

Sea:

$$r_n = \mathbf{w}^\top \phi(\mathbf{x}_n) - t_n.$$

Entonces:

$$E_n(\mathbf{w}) = \frac{1}{2} r_n^2.$$

Por regla de la cadena:

$$\nabla_{\mathbf{w}} E_n = r_n \nabla_{\mathbf{w}} r_n.$$

Como:

$$\nabla_{\mathbf{w}} r_n = \phi(\mathbf{x}_n),$$

obtenemos:

$$\nabla_{\mathbf{w}} E_n = \{ \mathbf{w}^\top \phi(\mathbf{x}_n) - t_n \} \phi(\mathbf{x}_n).$$

Ejemplo: Gradiente de $J(\mathbf{w})$

Sumando sobre todos los datos:

$$\nabla_{\mathbf{w}} \frac{1}{2} \sum_{n=1}^N \{ \mathbf{w}^\top \phi(\mathbf{x}_n) - t_n \}^2 = \sum_{n=1}^N \{ \mathbf{w}^\top \phi(\mathbf{x}_n) - t_n \} \phi(\mathbf{x}_n).$$

Además:

$$\nabla_{\mathbf{w}} \frac{\lambda}{2} \mathbf{w}^\top \mathbf{w} = \lambda \mathbf{w}.$$

Por tanto:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \sum_{n=1}^N \{ \mathbf{w}^\top \phi(\mathbf{x}_n) - t_n \} \phi(\mathbf{x}_n) + \lambda \mathbf{w}.$$

Ejemplo: Condición de óptimo

En el mínimo:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = 0.$$

Entonces:

$$\sum_{n=1}^N \{ \mathbf{w}^\top \phi(\mathbf{x}_n) - t_n \} \phi(\mathbf{x}_n) + \lambda \mathbf{w} = 0.$$

Reordenando:

$$\lambda \mathbf{w} = - \sum_{n=1}^N \{ \mathbf{w}^\top \phi(\mathbf{x}_n) - t_n \} \phi(\mathbf{x}_n).$$

Dividiendo por λ :

$$\mathbf{w} = - \frac{1}{\lambda} \sum_{n=1}^N \{ \mathbf{w}^\top \phi(\mathbf{x}_n) - t_n \} \phi(\mathbf{x}_n).$$

Ejemplo: Definición de los coeficientes duales

Definimos:

$$a_n = -\frac{1}{\lambda} \{ \mathbf{w}^\top \phi(\mathbf{x}_n) - t_n \}.$$

Entonces:

$$\mathbf{w} = \sum_{n=1}^N a_n \phi(\mathbf{x}_n).$$

Idea

Cada coeficiente a_n mide cuánto contribuye el dato $\mathbf{x}_n$ a la solución final. Así, la solución se describe mediante pesos asociados a datos, en lugar de coordenadas de características como se vio anteriormente.

Forma matricial de la representación dual

Sea:

$$\mathbf{a} = (a_1, \dots, a_N)^\top.$$

Recordando que:

$$\Phi = \begin{pmatrix} \phi(\mathbf{x}_1)^\top \\ \vdots \\ \phi(\mathbf{x}_N)^\top \end{pmatrix},$$

se tiene:

$$\mathbf{w} = \Phi^\top \mathbf{a}.$$

Dimensiones:

$$\Phi \in \mathbb{R}^{N \times M}, \quad \Phi^\top \in \mathbb{R}^{M \times N}, \quad \mathbf{a} \in \mathbb{R}^N,$$

por lo que:

$$\mathbf{w} \in \mathbb{R}^M.$$

Matriz de Gram

Definimos:

$$\mathbf{K} = \Phi\Phi^\top.$$

El elemento (n, m) es:

$$K_{nm} = \phi(\mathbf{x}_n)^\top \phi(\mathbf{x}_m).$$

Idea

$\mathbf{K}$ contiene todas las similitudes entre pares de datos en el espacio de características.

$$\mathbf{K} \in \mathbb{R}^{N \times N}.$$

Sustitución: predicciones usando $\mathbf{a}$

Como:

$$\mathbf{w} = \Phi^\top \mathbf{a},$$

entonces el vector de predicciones sobre entrenamiento es:

$$\mathbf{y} = \Phi \mathbf{w}.$$

Sustituyendo:

$$\mathbf{y} = \Phi \Phi^\top \mathbf{a}.$$

Pero:

$$\mathbf{K} = \Phi \Phi^\top.$$

Así:

$$\mathbf{y} = \mathbf{K} \mathbf{a}.$$

Por tanto, el error o residuo es:

$$\Phi \mathbf{w} - \mathbf{t} = \mathbf{K} \mathbf{a} - \mathbf{t}.$$

Sustitución: regularizador en forma dual

El término de regularización es:

$$\mathbf{w}^\top \mathbf{w}.$$

Usamos:

$$\mathbf{w} = \Phi^\top \mathbf{a}.$$

Entonces:

$$\mathbf{w}^\top \mathbf{w} = (\Phi^\top \mathbf{a})^\top (\Phi^\top \mathbf{a}).$$

Como:

$$(\Phi^\top \mathbf{a})^\top = \mathbf{a}^\top \Phi,$$

obtenemos:

$$\mathbf{w}^\top \mathbf{w} = \mathbf{a}^\top \Phi \Phi^\top \mathbf{a}.$$

Finalmente:

$$\mathbf{w}^\top \mathbf{w} = \mathbf{a}^\top \mathbf{K} \mathbf{a}.$$

Funcional de error dual

Partimos de:

$$J(\mathbf{w}) = \frac{1}{2} \|\Phi \mathbf{w} - \mathbf{t}\|^2 + \frac{\lambda}{2} \mathbf{w}^\top \mathbf{w}.$$

Ya vimos que:

$$\Phi \mathbf{w} = \mathbf{K} \mathbf{a}, \quad \mathbf{w}^\top \mathbf{w} = \mathbf{a}^\top \mathbf{K} \mathbf{a}.$$

Por tanto:

$$J(\mathbf{a}) = \frac{1}{2} (\mathbf{K} \mathbf{a} - \mathbf{t})^\top (\mathbf{K} \mathbf{a} - \mathbf{t}) + \frac{\lambda}{2} \mathbf{a}^\top \mathbf{K} \mathbf{a}.$$

Idea

La función objetivo ya solo depende de $\mathbf{a}$, $\mathbf{t}$ y $\mathbf{K}$.

Expansión de $J(\mathbf{a})$

Expandimos:

$$(\mathbf{K}\mathbf{a} - \mathbf{t})^\top (\mathbf{K}\mathbf{a} - \mathbf{t}) = (\mathbf{K}\mathbf{a})^\top (\mathbf{K}\mathbf{a}) - 2\mathbf{t}^\top \mathbf{K}\mathbf{a} + \mathbf{t}^\top \mathbf{t}.$$

Como $\mathbf{K} = \mathbf{K}^\top$:

$$(\mathbf{K}\mathbf{a})^\top (\mathbf{K}\mathbf{a}) = \mathbf{a}^\top \mathbf{K}\mathbf{K}\mathbf{a}.$$

Entonces:

$$J(\mathbf{a}) = \frac{1}{2}\mathbf{a}^\top \mathbf{K}\mathbf{K}\mathbf{a} - \mathbf{a}^\top \mathbf{K}\mathbf{t} + \frac{1}{2}\mathbf{t}^\top \mathbf{t} + \frac{\lambda}{2}\mathbf{a}^\top \mathbf{K}\mathbf{a}.$$

Gradiente respecto a $\mathbf{a}$

Derivamos término por término:

$$\nabla_{\mathbf{a}} \left(\frac{1}{2} \mathbf{a}^\top \mathbf{K} \mathbf{K} \mathbf{a} \right) = \mathbf{K} \mathbf{K} \mathbf{a}.$$

También:

$$\nabla_{\mathbf{a}} (-\mathbf{a}^\top \mathbf{K} \mathbf{t}) = -\mathbf{K} \mathbf{t}.$$

Y:

$$\nabla_{\mathbf{a}} \left(\frac{\lambda}{2} \mathbf{a}^\top \mathbf{K} \mathbf{a} \right) = \lambda \mathbf{K} \mathbf{a}.$$

Por tanto:

$$\nabla_{\mathbf{a}} J(\mathbf{a}) = \mathbf{K} \mathbf{K} \mathbf{a} - \mathbf{K} \mathbf{t} + \lambda \mathbf{K} \mathbf{a}.$$

Ecuación normal dual

Tenemos:

$$\nabla_{\mathbf{a}} J(\mathbf{a}) = \mathbf{K}\mathbf{K}\mathbf{a} - \mathbf{K}\mathbf{t} + \lambda\mathbf{K}\mathbf{a}.$$

Factorizamos $\mathbf{K}$:

$$\nabla_{\mathbf{a}} J(\mathbf{a}) = \mathbf{K}(\mathbf{K}\mathbf{a} - \mathbf{t} + \lambda\mathbf{a}).$$

Equivalente:

$$\nabla_{\mathbf{a}} J(\mathbf{a}) = \mathbf{K}\{(\mathbf{K} + \lambda\mathbf{I})\mathbf{a} - \mathbf{t}\}.$$

Para minimizar, una solución natural satisface:

$$(\mathbf{K} + \lambda\mathbf{I})\mathbf{a} = \mathbf{t}.$$

Solución dual

De:

$$(\mathbf{K} + \lambda \mathbf{I})\mathbf{a} = \mathbf{t},$$

obtenemos:

$$\mathbf{a} = (\mathbf{K} + \lambda \mathbf{I})^{-1}\mathbf{t}.$$

Idea

En lugar de estimar directamente $\mathbf{w}$, estimamos coeficientes sobre los datos.

Nota sobre λ

El término $\lambda \mathbf{I}$ mejora la estabilidad numérica y controla la complejidad de la solución.

Predicción para un nuevo punto

Para un nuevo punto $\mathbf{x}$:

$$y(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}).$$

Usamos la representación dual:

$$\mathbf{w} = \Phi^\top \mathbf{a}.$$

Entonces:

$$y(\mathbf{x}) = (\Phi^\top \mathbf{a})^\top \phi(\mathbf{x}).$$

Como:

$$(\Phi^\top \mathbf{a})^\top = \mathbf{a}^\top \Phi,$$

tenemos:

$$y(\mathbf{x}) = \mathbf{a}^\top \Phi \phi(\mathbf{x}).$$

Aparece el vector kernel

El vector:

$$\Phi\phi(\mathbf{x})$$

tiene componentes:

$$\begin{pmatrix} \phi(\mathbf{x}_1)^\top \phi(\mathbf{x}) \\ \phi(\mathbf{x}_2)^\top \phi(\mathbf{x}) \\ \vdots \\ \phi(\mathbf{x}_N)^\top \phi(\mathbf{x}) \end{pmatrix}.$$

Definimos:

$$k(\mathbf{x}_n, \mathbf{x}) = \phi(\mathbf{x}_n)^\top \phi(\mathbf{x}).$$

Por tanto:

$$\mathbf{k}(\mathbf{x}) = \begin{pmatrix} k(\mathbf{x}_1, \mathbf{x}) \\ \vdots \\ k(\mathbf{x}_N, \mathbf{x}) \end{pmatrix}.$$

Fórmula dual de predicción

Como:

$$y(\mathbf{x}) = \mathbf{a}^\top \mathbf{k}(\mathbf{x}),$$

también escribimos:

$$y(\mathbf{x}) = \mathbf{k}(\mathbf{x})^\top \mathbf{a}.$$

Usando:

$$\mathbf{a} = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{t},$$

obtenemos:

$$y(\mathbf{x}) = \mathbf{k}(\mathbf{x})^\top (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{t}$$

Idea

La predicción depende de las similitudes entre $\mathbf{x}$ y los datos de entrenamiento.

Kernel trick

Si todas las expresiones dependen de $\phi(\mathbf{x})$ únicamente mediante productos internos:

$$\phi(\mathbf{x})^\top \phi(\mathbf{x}'),$$

podemos sustituirlos por una función:

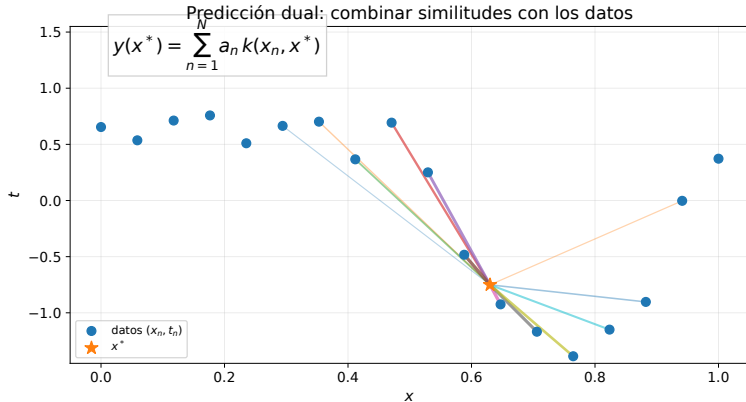
$$k(\mathbf{x}, \mathbf{x}').$$

Kernel trick

Trabajamos con $k(\mathbf{x}, \mathbf{x}')$ sin construir explícitamente $\phi(\mathbf{x})$.

Esto permite usar espacios de características muy grandes o incluso infinitos.

Figura: predicción dual por similitudes



Idea

Un nuevo punto se predice comparándolo con todos los datos de entrenamiento mediante $k(\mathbf{x}_n, \mathbf{x})$.

Primal vs dual

Forma primal de regresión lineal con regularización cuadrática (ridge regression):

$$\mathbf{w} = (\Phi^\top \Phi + \lambda \mathbf{I})^{-1} \Phi^\top \mathbf{t}.$$

Forma dual:

$$\mathbf{a} = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{t}.$$

Predicción dual:

$$y(\mathbf{x}) = \mathbf{k}(\mathbf{x})^\top \mathbf{a}.$$

Idea

La forma primal trabaja con parámetros de características. La forma dual trabaja con coeficientes asociados a datos.

Ejemplo en R: primal y dual

Objetivo: verificar numéricamente que ambas formulaciones pueden dar la misma predicción cuando usamos las mismas bases.

```
# Primal:  $w = (\Phi^T \Phi + \lambda I)^{-1} \Phi^T t$ 
w <- solve(t(Phi) %*% Phi + lambda * diag(M), t(Phi) %*% t)
y_primal <- Phi_grid %*% w

# Dual:  $a = (K + \lambda I)^{-1} t$ 
K <- Phi %*% t(Phi)
a <- solve(K + lambda * diag(N), t)
y_dual <- K_grid %*% a
```

Definición de kernel

Una función $k(\mathbf{x}, \mathbf{x}')$ es un kernel si existe una transformación ϕ tal que:

$$k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}').$$

Propiedad inmediata:

$$k(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}', \mathbf{x}),$$

porque:

$$\phi(\mathbf{x})^\top \phi(\mathbf{x}') = \phi(\mathbf{x}')^\top \phi(\mathbf{x}).$$

Nota

Simetría no basta. También necesitamos positividad semidefinida de la matriz de Gram.

Kernel lineal

Si elegimos:

$$\phi(\mathbf{x}) = \mathbf{x},$$

entonces:

$$k(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}'.$$

Idea

Este kernel no cambia el espacio. Solo mide producto interno en el espacio original.

Es el punto de referencia para entender kernels más complejos.

Kernel polinomial de grado 2

Para:

$$\mathbf{x} = (x_1, x_2)^\top, \quad \mathbf{z} = (z_1, z_2)^\top,$$

definimos:

$$k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z})^2.$$

Como:

$$\mathbf{x}^\top \mathbf{z} = x_1 z_1 + x_2 z_2,$$

entonces:

$$k(\mathbf{x}, \mathbf{z}) = (x_1 z_1 + x_2 z_2)^2.$$

Expansión del kernel polinomial

Desarrollamos:

$$(x_1 z_1 + x_2 z_2)^2 = x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2.$$

Reescribimos:

$$= (x_1^2)(z_1^2) + (\sqrt{2}x_1 x_2)(\sqrt{2}z_1 z_2) + (x_2^2)(z_2^2).$$

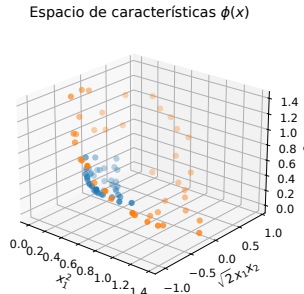
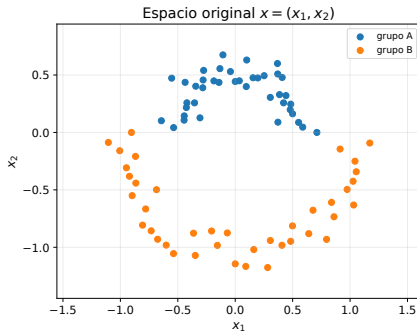
Por tanto:

$$k(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\top \phi(\mathbf{z}),$$

con:

$$\phi(\mathbf{x}) = \begin{pmatrix} x_1^2 \\ \sqrt{2}x_1 x_2 \\ x_2^2 \end{pmatrix}.$$

Figura: mapeo polinomial



Interpretación

El kernel polinomial calcula un producto interno en un espacio donde aparecen términos no lineales como x_1^2 , x_1x_2 y x_2^2 .

Matriz de Gram - Sustitución con función kernel

Dado un conjunto de puntos:

$$\mathbf{x}_1, \dots, \mathbf{x}_N,$$

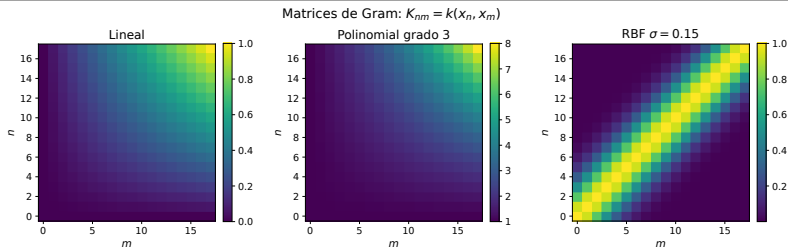
definimos:

$$K_{nm} = k(\mathbf{x}_n, \mathbf{x}_m).$$

La matriz de Gram queda:

$$\mathbf{K} = \begin{pmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{pmatrix}$$

Figura: matrices de Gram



Colores más claros indican valores grandes de K_{nm} , es decir, mayor similitud entre x_n y x_m . Colores más oscuros indican valores pequeños, es decir, menor similitud. En el kernel lineal y polinomial, los valores grandes aparecen cuando los productos internos entre puntos son grandes. En el kernel RBF, la diagonal es clara porque cada punto es idéntico a sí mismo: $k(x_n, x_n) = 1$. Además, los puntos cercanos entre sí producen valores altos, por eso aparece una banda clara alrededor de la diagonal.

Condición PSD

Para que k sea un kernel válido, $\mathbf{K}$ debe ser semidefinida positiva para cualquier conjunto de puntos.

Esto significa:

$$\mathbf{a}^\top \mathbf{K} \mathbf{a} \geq 0, \quad \text{para todo } \mathbf{a} \in \mathbb{R}^N.$$

Intuición

Si $\mathbf{K}$ viene de productos internos, entonces no puede producir una “longitud cuadrada” negativa.

Por qué la matriz de Gram es PSD

Si:

$$\mathbf{K} = \Phi\Phi^\top,$$

entonces, para cualquier $\mathbf{a}$:

$$\mathbf{a}^\top \mathbf{K} \mathbf{a} = \mathbf{a}^\top \Phi\Phi^\top \mathbf{a}.$$

Agrupando:

$$\mathbf{a}^\top \Phi\Phi^\top \mathbf{a} = (\Phi^\top \mathbf{a})^\top (\Phi^\top \mathbf{a}).$$

Por tanto:

$$\mathbf{a}^\top \mathbf{K} \mathbf{a} = \|\Phi^\top \mathbf{a}\|^2 \geq 0.$$

Conclusión

Todo kernel válido genera matrices de Gram semidefinidas positivas.

Eigenvalores y validez de kernels

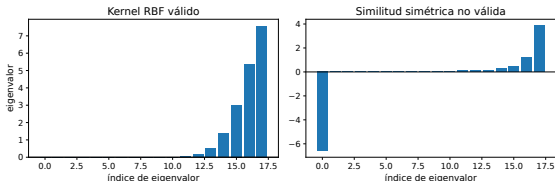
Una matriz simétrica es semidefinida positiva si sus eigenvalores son no negativos:

$$\lambda_i(\mathbf{K}) \geq 0.$$

Criterio práctico

Si para ciertos datos una función de similitud genera una matriz de Gram con eigenvalores negativos, esa función no puede ser un kernel válido para todos los conjuntos de datos.

Criterio PSD: los eigenvalores de K deben ser no negativos



Reglas de construcción de kernels

Si k_1 y k_2 son kernels válidos, entonces bajo condiciones apropiadas también lo son:

$$k(\mathbf{x}, \mathbf{x}') = c k_1(\mathbf{x}, \mathbf{x}'), \quad c > 0,$$

$$k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') + k_2(\mathbf{x}, \mathbf{x}'),$$

$$k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') k_2(\mathbf{x}, \mathbf{x}').$$

Demostración: suma de kernels

Si $\mathbf{K}_1$ y $\mathbf{K}_2$ son PSD, entonces:

$$\mathbf{a}^\top (\mathbf{K}_1 + \mathbf{K}_2) \mathbf{a} = \mathbf{a}^\top \mathbf{K}_1 \mathbf{a} + \mathbf{a}^\top \mathbf{K}_2 \mathbf{a}.$$

Como:

$$\mathbf{a}^\top \mathbf{K}_1 \mathbf{a} \geq 0, \quad \mathbf{a}^\top \mathbf{K}_2 \mathbf{a} \geq 0,$$

se obtiene:

$$\mathbf{a}^\top (\mathbf{K}_1 + \mathbf{K}_2) \mathbf{a} \geq 0.$$

Por tanto:

$$k_1 + k_2$$

es kernel válido.

Demostración: constante positiva

Si $\mathbf{K}_1$ es PSD y $c > 0$, definimos:

$$\mathbf{K} = c\mathbf{K}_1.$$

Entonces:

$$\mathbf{a}^\top \mathbf{K} \mathbf{a} = \mathbf{a}^\top (c\mathbf{K}_1) \mathbf{a} = c \mathbf{a}^\top \mathbf{K}_1 \mathbf{a}.$$

Como:

$$c > 0, \quad \mathbf{a}^\top \mathbf{K}_1 \mathbf{a} \geq 0,$$

se sigue que:

$$\mathbf{a}^\top \mathbf{K} \mathbf{a} \geq 0.$$

Por tanto:

$$ck_1$$

es kernel válido.

Kernel polinomial general

Una familia importante es:

$$k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + c)^M.$$

- $M = 1$: términos lineales.
- $M = 2$: términos cuadráticos e interacciones pares.
- $M = 3$: términos cúbicos e interacciones de tercer orden.

Idea

El grado M controla el orden de interacciones entre variables que quedan representadas implícitamente.

Kernel gaussiano o RBF

El kernel gaussiano se define como:

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right).$$

Idea

$$\mathbf{x} \approx \mathbf{x}' \implies k(\mathbf{x}, \mathbf{x}') \approx 1.$$

$$\mathbf{x} \text{ lejos de } \mathbf{x}' \implies k(\mathbf{x}, \mathbf{x}') \approx 0.$$

El parámetro σ define la escala de vecindad.

Expansión de la distancia

Partimos de:

$$\|\mathbf{x} - \mathbf{x}'\|^2 = (\mathbf{x} - \mathbf{x}')^\top (\mathbf{x} - \mathbf{x}').$$

Desarrollando:

$$\|\mathbf{x} - \mathbf{x}'\|^2 = \mathbf{x}^\top \mathbf{x} - 2\mathbf{x}^\top \mathbf{x}' + (\mathbf{x}')^\top \mathbf{x}'.$$

Sustituyendo:

$$k(\mathbf{x}, \mathbf{x}') = \exp \left(-\frac{\mathbf{x}^\top \mathbf{x} - 2\mathbf{x}^\top \mathbf{x}' + (\mathbf{x}')^\top \mathbf{x}'}{2\sigma^2} \right).$$

Factorización del kernel gaussiano

Usando propiedades de la exponencial:

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\mathbf{x}^\top \mathbf{x}}{2\sigma^2}\right) \exp\left(\frac{\mathbf{x}^\top \mathbf{x}'}{\sigma^2}\right) \exp\left(-\frac{(\mathbf{x}')^\top \mathbf{x}'}{2\sigma^2}\right).$$

Interpretación

El término central depende del producto interno $\mathbf{x}^\top \mathbf{x}'$. La exponencial puede entenderse como una suma infinita de potencias del producto interno; por eso se relaciona con un espacio de características de dimensión infinita.

Ejemplo: kernel gaussiano

Algunos kernels equivalen a calcular un producto interno en un espacio de características enorme, aunque nunca escribamos explícitamente ese espacio.

Por ejemplo, el kernel gaussiano

$$k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$$

puede interpretarse como un producto interno que involucra infinitas características, relacionadas con potencias de los componentes de x . La ventaja del kernel trick es que no necesitamos construir el vector

$$\phi(x) = (\phi_1(x), \phi_2(x), \dots)$$

ni almacenar todas sus componentes. Basta calcular directamente el número

$$k(x, x') = \phi(x)^\top \phi(x').$$

Ejemplo: características implícitas del kernel gaussiano

En una dimensión, el kernel gaussiano puede escribirse como

$$k(x, x') = \exp\left(-\frac{x^2}{2\sigma^2}\right) \exp\left(-\frac{(x')^2}{2\sigma^2}\right) \sum_{m=0}^{\infty} \frac{x^m (x')^m}{m! \sigma^{2m}}.$$

Por tanto,

$$k(x, x') = \sum_{m=0}^{\infty} \phi_m(x) \phi_m(x'),$$

con

$$\phi_m(x) = \exp\left(-\frac{x^2}{2\sigma^2}\right) \frac{x^m}{\sigma^m \sqrt{m!}}.$$

Así, el kernel gaussiano equivale a un producto interno entre vectores con infinitas componentes:

$$\phi(x) = \exp\left(-\frac{x^2}{2\sigma^2}\right) \left(1, \frac{x}{\sigma}, \frac{x^2}{\sigma^2 \sqrt{2!}}, \frac{x^3}{\sigma^3 \sqrt{3!}}, \dots\right)^{\top}.$$

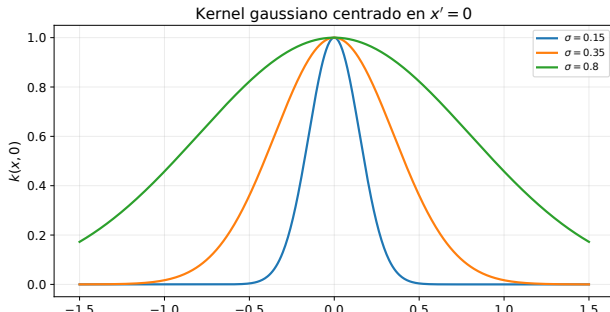
Efecto de σ

En:

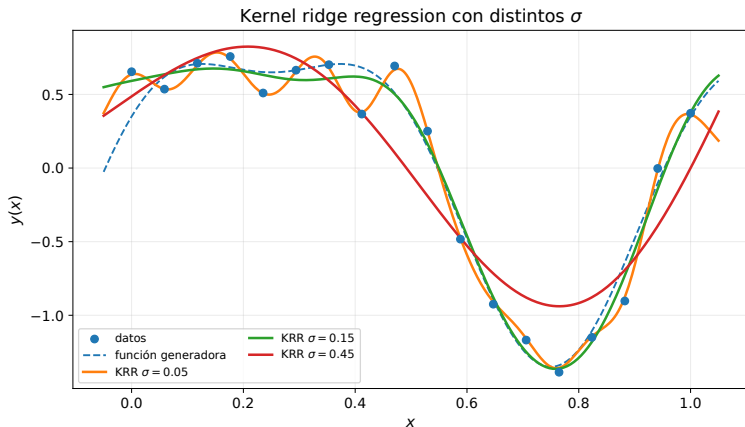
$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right),$$

σ controla qué tan rápido cae la similitud con la distancia.

- σ pequeño: influencia muy local.
- σ grande: influencia más global.



Ejemplo en R: kernel ridge con distintos σ



Idea

El ancho del kernel controla el balance entre modelos demasiado variables y modelos demasiado suaves.

Otros kernels...

Bishop también menciona kernels sobre:

- conjuntos,
- objetos estructurados,
- distribuciones de probabilidad,
- modelos generativos y Fisher kernels.

Nota

Estos temas se dejan para investigación posterior. Para este primer acercamiento, nos concentramos en kernels como productos internos implícitos y en su uso para regresión.

Conclusión Parte 1

La idea principal es:

$$J(\mathbf{w}) \implies \mathbf{w} = \Phi^\top \mathbf{a} \implies \mathbf{K} = \Phi \Phi^\top.$$

La predicción queda:

$$y(\mathbf{x}) = \mathbf{k}(\mathbf{x})^\top (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{t}$$

Resumen

Un método kernel predice usando similitudes entre el nuevo punto y los datos de entrenamiento.

De kernels a funciones de base radial

Ahora conectamos kernels con modelos que usan funciones centradas en puntos.

predicción = combinación de respuestas locales.

Idea central

Una función de base radial responde según la distancia entre $\mathbf{x}$ y un centro μ_j .

Función de base radial

Una función radial depende solo de la distancia a un centro:

$$\phi_j(\mathbf{x}) = h(\|\mathbf{x} - \boldsymbol{\mu}_j\|).$$

Aquí:

$$\boldsymbol{\mu}_j$$

es el centro de la función base.

Interpretación

La base responde fuerte cerca de $\boldsymbol{\mu}_j$ y débil lejos de $\boldsymbol{\mu}_j$.

Red RBF: forma general

Una red RBF tiene la forma:

$$y(\mathbf{x}) = \sum_{j=1}^M w_j h(\|\mathbf{x} - \boldsymbol{\mu}_j\|).$$

Comparación con MLP

En un MLP, las bases se construyen mediante pesos internos y activaciones.
En una RBF network, las bases están localizadas alrededor de centros.

Base gaussiana radial

Una elección común es:

$$h(r) = \exp\left(-\frac{r^2}{2\sigma^2}\right).$$

Entonces:

$$h(\|\mathbf{x} - \boldsymbol{\mu}_j\|) = \exp\left(-\frac{\|\mathbf{x} - \boldsymbol{\mu}_j\|^2}{2\sigma^2}\right).$$

Idea

Cada centro genera una campana local. La salida final combina esas campanas con pesos w_j .

Caso especial: un centro en cada dato

Si tomamos:

$$\mu_n = \mathbf{x}_n,$$

entonces:

$$y(\mathbf{x}) = \sum_{n=1}^N w_n h(\|\mathbf{x} - \mathbf{x}_n\|).$$

Idea

Este caso conecta las RBF networks con modelos basados en memoria: cada dato de entrenamiento aporta una función base centrada en sí mismo. En una RBF network, los valores objetivo t_n no aparecen directamente en la predicción; se usan durante el entrenamiento para encontrar los pesos w_n .

Interpolación exacta con RBF

Queremos que el modelo pase por los datos:

$$y(\mathbf{x}_m) = t_m, \quad m = 1, \dots, N.$$

Sustituyendo:

$$\sum_{n=1}^N w_n h(\|\mathbf{x}_m - \mathbf{x}_n\|) = t_m.$$

Definimos:

$$H_{mn} = h(\|\mathbf{x}_m - \mathbf{x}_n\|).$$

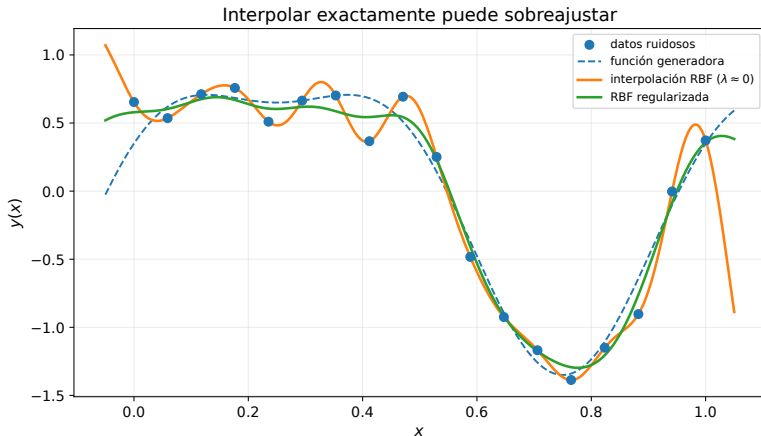
Entonces el sistema es:

$$\mathbf{H}\mathbf{w} = \mathbf{t}.$$

Si $\mathbf{H}$ es invertible:

$$\mathbf{w} = \mathbf{H}^{-1}\mathbf{t}.$$

Figura: interpolación vs regularización



Idea

La interpolación exacta puede sobreajustar datos con ruido.

Menos centros que datos

La regularización suaviza la solución. En vez de usar N centros, podemos usar $M < N$:

$$y(\mathbf{x}) = \sum_{j=1}^M w_j h(\|\mathbf{x} - \boldsymbol{\mu}_j\|).$$

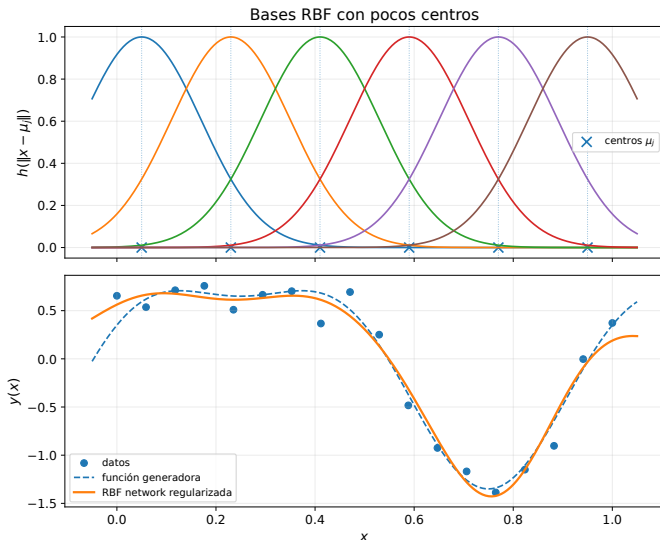
Los centros pueden elegirse:

- aleatoriamente,
- mediante k -means,
- por selección incremental,
- usando validación.

Idea

Menos centros reducen la complejidad (regularización implícita) y el costo computacional.

Figura: centros RBF y ajuste



RBF regularizada explícitamente

Podemos estimar $\mathbf{w}$ minimizando:

$$J(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \left[\sum_{j=1}^M w_j h(\|\mathbf{x}_n - \boldsymbol{\mu}_j\|) - t_n \right]^2 + \frac{\lambda}{2} \mathbf{w}^\top \mathbf{w}.$$

Idea

Una RBF network regularizada es regresión lineal regularizada sobre bases radiales.

Bases RBF normalizadas

Definimos:

$$\tilde{h}_n(\mathbf{x}) = \frac{h(\|\mathbf{x} - \mathbf{x}_n\|)}{\sum_{m=1}^N h(\|\mathbf{x} - \mathbf{x}_m\|)}.$$

Entonces:

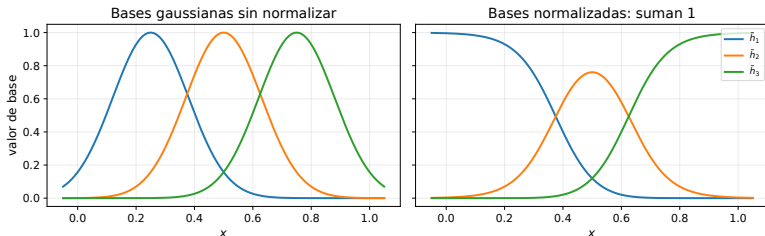
$$\sum_{n=1}^N \tilde{h}_n(\mathbf{x}) = \sum_{n=1}^N \frac{h(\|\mathbf{x} - \mathbf{x}_n\|)}{\sum_{m=1}^N h(\|\mathbf{x} - \mathbf{x}_m\|)}.$$

Como el denominador no depende de n :

$$\sum_{n=1}^N \tilde{h}_n(\mathbf{x}) = \frac{\sum_{n=1}^N h(\|\mathbf{x} - \mathbf{x}_n\|)}{\sum_{m=1}^N h(\|\mathbf{x} - \mathbf{x}_m\|)} = 1.$$

Figura: bases normalizadas y no normalizadas

$$\tilde{h}_n(x) = h_n(x) / \sum_m h_m(x)$$



Idea

La normalización convierte respuestas locales en pesos que suman uno.

Predicción como promedio ponderado

Si usamos bases normalizadas:

$$y(\mathbf{x}) = \sum_{n=1}^N \tilde{h}_n(\mathbf{x}) t_n.$$

Como:

$$\tilde{h}_n(\mathbf{x}) \geq 0, \quad \sum_{n=1}^N \tilde{h}_n(\mathbf{x}) = 1,$$

la predicción es un promedio ponderado local.

Nota

Esta es la forma que llevará directamente al modelo de Nadaraya–Watson.

De RBF normalizada a Nadaraya–Watson

El modelo de Nadaraya–Watson tiene la forma:

$$y(\mathbf{x}) = \sum_{n=1}^N k(\mathbf{x}, \mathbf{x}_n) t_n,$$

donde:

$$k(\mathbf{x}, \mathbf{x}_n) \geq 0, \quad \sum_{n=1}^N k(\mathbf{x}, \mathbf{x}_n) = 1.$$

Pregunta

¿Cómo aparece esta expresión desde una interpretación probabilística?

Objetivo probabilístico

En regresión, una predicción natural es la esperanza condicional:

$$y(\mathbf{x}) = \mathbb{E}[t \mid \mathbf{x}].$$

Por definición:

$$\mathbb{E}[t \mid \mathbf{x}] = \int t p(t \mid \mathbf{x}) dt.$$

Así que si aproximamos $p(t \mid \mathbf{x})$, podemos obtener una predicción para t .

Densidad conjunta tipo Parzen

Aproximamos la densidad conjunta con componentes centradas en los datos:

$$p(\mathbf{x}, t) = \frac{1}{N} \sum_{n=1}^N f(\mathbf{x} - \mathbf{x}_n, t - t_n).$$

Idea

Cada dato $(\mathbf{x}_n, t_n)$ aporta una pequeña nube de probabilidad alrededor de sí mismo.

De densidad conjunta a condicional

Usamos:

$$p(t \mid \mathbf{x}) = \frac{p(\mathbf{x}, t)}{p(\mathbf{x})}.$$

Además:

$$p(\mathbf{x}) = \int p(\mathbf{x}, t) dt.$$

Sustituyendo la aproximación conjunta:

$$p(t \mid \mathbf{x}) = \frac{\sum_{n=1}^N f(\mathbf{x} - \mathbf{x}_n, t - t_n)}{\sum_{m=1}^N \int f(\mathbf{x} - \mathbf{x}_m, t - t_m) dt}.$$

El factor $1/N$ se cancela.

Sustitución en la esperanza condicional

Partimos de:

$$y(\mathbf{x}) = \int t p(t \mid \mathbf{x}) dt.$$

Sustituyendo:

$$y(\mathbf{x}) = \frac{\sum_{n=1}^N \int t f(\mathbf{x} - \mathbf{x}_n, t - t_n) dt}{\sum_{m=1}^N \int f(\mathbf{x} - \mathbf{x}_m, t - t_m) dt}.$$

Idea

Haremos un cambio de variable para separar el efecto de t_n .

Cambio de variable

Para cada término definimos:

$$u = t - t_n, \quad t = u + t_n.$$

Entonces:

$$\int t f(\mathbf{x} - \mathbf{x}_n, t - t_n) dt = \int (u + t_n) f(\mathbf{x} - \mathbf{x}_n, u) du.$$

Separando:

$$= \int u f(\mathbf{x} - \mathbf{x}_n, u) du + t_n \int f(\mathbf{x} - \mathbf{x}_n, u) du.$$

Supuesto de media cero

Suponemos que las componentes tienen media cero en la dirección de t :

$$\int u f(\mathbf{x} - \mathbf{x}_n, u) du = 0.$$

Entonces:

$$\int t f(\mathbf{x} - \mathbf{x}_n, t - t_n) dt = t_n \int f(\mathbf{x} - \mathbf{x}_n, u) du.$$

Definimos:

$$g(\mathbf{x} - \mathbf{x}_n) = \int f(\mathbf{x} - \mathbf{x}_n, u) du.$$

Por tanto:

$$\int t f(\mathbf{x} - \mathbf{x}_n, t - t_n) dt = t_n g(\mathbf{x} - \mathbf{x}_n).$$

Modelo de Nadaraya–Watson

Sustituyendo en la esperanza condicional:

$$y(\mathbf{x}) = \frac{\sum_{n=1}^N g(\mathbf{x} - \mathbf{x}_n) t_n}{\sum_{m=1}^N g(\mathbf{x} - \mathbf{x}_m)}.$$

Definimos pesos normalizados:

$$k(\mathbf{x}, \mathbf{x}_n) = \frac{g(\mathbf{x} - \mathbf{x}_n)}{\sum_{m=1}^N g(\mathbf{x} - \mathbf{x}_m)}.$$

Entonces:

$$y(\mathbf{x}) = \sum_{n=1}^N k(\mathbf{x}, \mathbf{x}_n) t_n$$

Los pesos suman uno

Verificamos:

$$\sum_{n=1}^N k(\mathbf{x}, \mathbf{x}_n) = \sum_{n=1}^N \frac{g(\mathbf{x} - \mathbf{x}_n)}{\sum_{m=1}^N g(\mathbf{x} - \mathbf{x}_m)}.$$

Como el denominador no depende de n :

$$= \frac{\sum_{n=1}^N g(\mathbf{x} - \mathbf{x}_n)}{\sum_{m=1}^N g(\mathbf{x} - \mathbf{x}_m)} = 1.$$

Conclusión

Nadaraya–Watson predice mediante un promedio ponderado local.

Kernel gaussiano en Nadaraya–Watson

Una elección común es:

$$g(\mathbf{x} - \mathbf{x}_n) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_n\|^2}{2\sigma^2}\right).$$

Entonces:

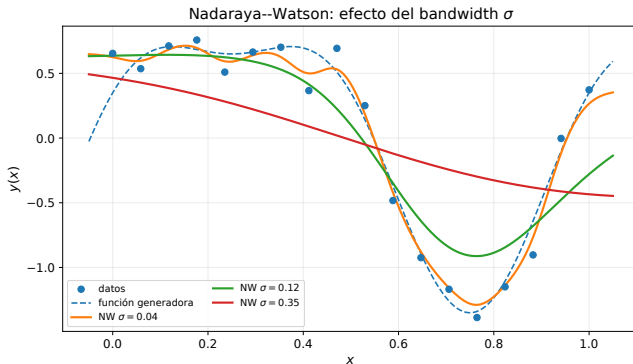
$$k(\mathbf{x}, \mathbf{x}_n) = \frac{\exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_n\|^2}{2\sigma^2}\right)}{\sum_{m=1}^N \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_m\|^2}{2\sigma^2}\right)}.$$

Idea

Los puntos cercanos a $\mathbf{x}$ reciben mayor peso.

Efecto del bandwidth σ

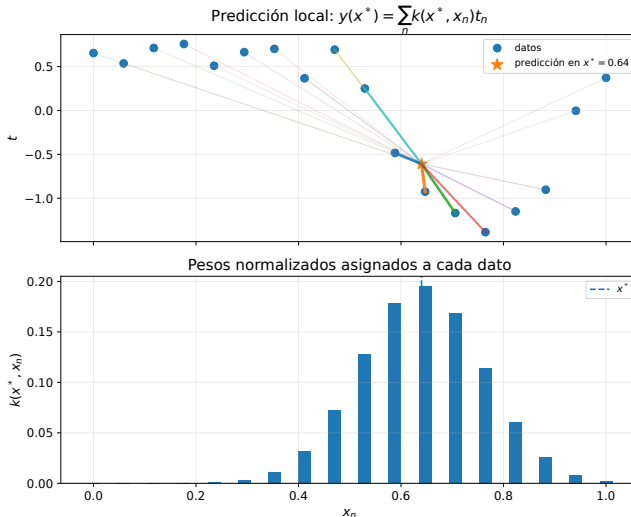
- Si σ es muy pequeño, $y(x)$ depende casi solo del dato más cercano.
- Si σ es muy grande, $y(x)$ se aproxima a un promedio global.



Idea

σ controla el balance sesgo-varianza.

Figura: pesos locales para un punto de prueba



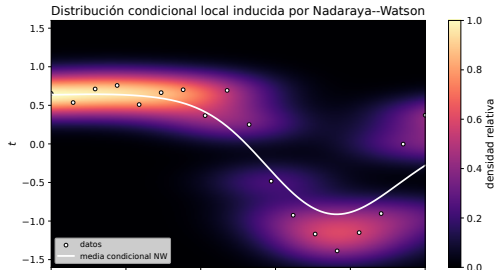
Distribución condicional completa

El procedimiento también induce una distribución condicional:

$$p(t \mid \mathbf{x}) = \frac{\sum_{n=1}^N f(\mathbf{x} - \mathbf{x}_n, t - t_n)}{\sum_{m=1}^N g(\mathbf{x} - \mathbf{x}_m)}.$$

Interpretación

Nadaraya–Watson no solo puede verse como una fórmula de predicción; también surge como media de una aproximación local a $p(t \mid \mathbf{x})$.



Ejemplo en R: Nadaraya–Watson

```
nw_predict <- function(x_grid, x_train, t_train, sigma) {  
  sapply(x_grid, function(x) {  
    weights <- exp(-(x - x_train)^2 / (2 * sigma^2))  
    weights <- weights / sum(weights)  
    sum(weights * t_train)  
  })  
}
```

Idea

La implementación refleja directamente la fórmula:

$$y(x) = \sum_n k(x, x_n) t_n.$$

RBF network vs Nadaraya–Watson

RBF network:

$$y(\mathbf{x}) = \sum_{j=1}^M w_j h(\|\mathbf{x} - \boldsymbol{\mu}_j\|).$$

Los pesos w_j se aprenden globalmente.

Nadaraya–Watson:

$$y(\mathbf{x}) = \sum_{n=1}^N k(\mathbf{x}, \mathbf{x}_n) t_n.$$

Los pesos $k(\mathbf{x}, \mathbf{x}_n)$ dependen del punto de prueba y se normalizan localmente.

Kernel ridge vs Nadaraya–Watson

Kernel ridge:

$$y(\mathbf{x}) = \mathbf{k}(\mathbf{x})^\top (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{t}.$$

Nadaraya–Watson:

$$y(\mathbf{x}) = \frac{\sum_n g(\mathbf{x} - \mathbf{x}_n) t_n}{\sum_m g(\mathbf{x} - \mathbf{x}_m)}.$$

Idea

Kernel ridge aprende coeficientes globales mediante un sistema lineal. Nadaraya–Watson usa pesos locales normalizados para cada punto nuevo. Considerando también RBF, los tres modelos usan funciones de similitud, pero difieren en cómo asignan pesos y en cómo controlan la complejidad.

Figura: comparación de tres métodos

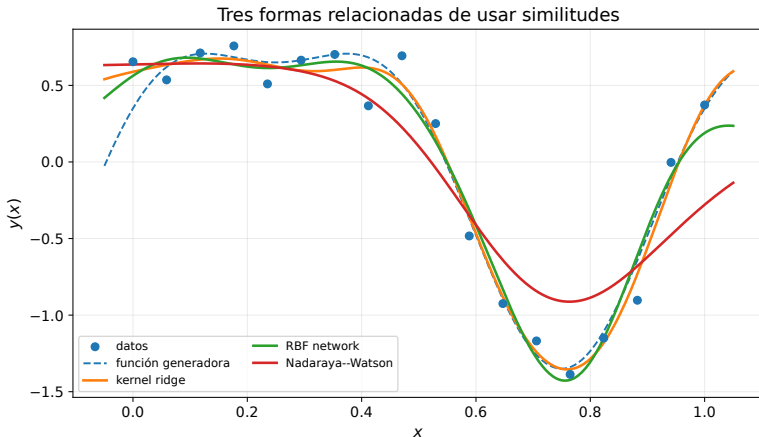
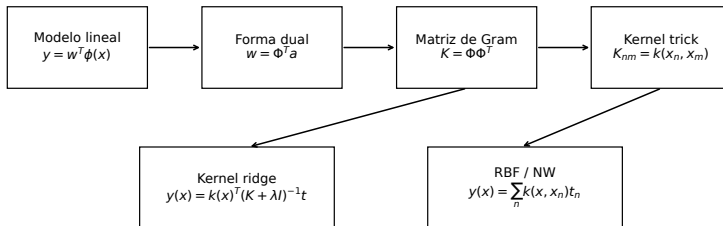


Figura resumen: flujo conceptual

Ruta conceptual de la presentación



Idea

La dualidad convierte el aprendizaje en un problema de similitudes; las RBF y Nadaraya–Watson muestran cómo esas similitudes producen predicciones locales.

Temas para estudio posterior

Para complementar a Bishop en una investigación posterior:

- 1 Teorema del representer.
- 2 Espacios RKHS.
- 3 Teorema de Mercer.
- 4 Diferencia entre similitud y kernel válido.
- 5 Validación cruzada para elegir σ y λ .
- 6 Relación entre kernel ridge regression y Gaussian Processes.

Resumen

El orden de ideas es:

$$\begin{aligned} y(\mathbf{x}, \mathbf{w}) &= \mathbf{w}^\top \phi(\mathbf{x}) \\ &\Downarrow \\ \mathbf{w} &= \Phi^\top \mathbf{a} \\ &\Downarrow \\ y(\mathbf{x}) &= \mathbf{k}(\mathbf{x})^\top (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{t} \\ &\Downarrow \\ y(\mathbf{x}) &= \sum_n k(\mathbf{x}, \mathbf{x}_n) t_n. \end{aligned}$$

Conclusión

Los métodos kernel convierten productos internos en herramientas de similitud, permitiendo construir modelos no lineales de manera estructurada.